DATABASE MANAGEMENT SYSTEMS LABORATORY

MySQL Command Line - Output Screenshots

Experiments 15 to 25

Ex. No.	Name of the Experiment
15	Simple Programming using CASE and LOOP
16	High Level Programming Extensions - Procedures
17	High Level Programming Extensions - Function
18	High Level Programming Extensions - Cursors
19	High Level Programming Extensions - Triggers
20	MySQL String Functions - REPLACE, REPEAT, REVERSE, RIGHT, LEFT, RPAD, LPAD
21	MySQL String Functions - SPACE, SUBSTR, UPPER, LOWER, TRIM, LENGTH
22	Database Connectivity Using PHP and MySQL
23	Case Scenario: Train Ticket Reservation System to Receive Tickets through Social Network
24	Case Scenario: College Admission Form
25	Case Scenario: QR Enabled Automatic Bus Ticket Booking System

Name : _____ Register No. : _____

Class : _____ Date : _____

Ex. No. 15

Simple Programming using CASE and LOOP

Aim: To write simple MySQL programs using the CASE statement / CASE expression and the LOOP construct (with LEAVE and ITERATE) inside stored procedures.

Output (MySQL Command Line):

MySQL Command Line - Ex. 15

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex15_case_loop;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex15_case_loop;
Database changed
mysql> CREATE TABLE student (rollno INT PRIMARY KEY, name VARCHAR(30), mark INT);
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO student VALUES (101,'Arun',92),(102,'Bala',78),(103,'Catherine',64),
    -> (104,'Dinesh',55),(105,'Elango',31);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM student;
+--------+-----------+------+
| rollno | name      | mark |
+--------+-----------+------+
|    101 | Arun      |   92 |
|    102 | Bala      |   78 |
|    103 | Catherine |   64 |
|    104 | Dinesh    |   55 |
|    105 | Elango    |   31 |
+--------+-----------+------+
5 rows in set (0.00 sec)
```

MySQL Command Line - Ex. 15

```
mysql> SELECT rollno, name, mark,
    -> CASE
    ->   WHEN mark >= 90 THEN 'O'
    ->   WHEN mark >= 75 THEN 'A'
    ->   WHEN mark >= 60 THEN 'B'
    ->   WHEN mark >= 50 THEN 'C'
    ->   ELSE 'Fail'
    -> END AS grade
    -> FROM student;
+--------+-----------+------+-------+
| rollno | name      | mark | grade |
+--------+-----------+------+-------+
|    101 | Arun      |   92 | O     |
|    102 | Bala      |   78 | A     |
|    103 | Catherine |   64 | B     |
|    104 | Dinesh    |   55 | C     |
|    105 | Elango    |   31 | Fail  |
+--------+-----------+------+-------+
5 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 15

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE day_of_week(IN n INT)
    -> BEGIN
    ->   DECLARE d VARCHAR(20);
    ->   CASE n
    ->     WHEN 1 THEN SET d = 'Monday';
    ->     WHEN 2 THEN SET d = 'Tuesday';
    ->     WHEN 3 THEN SET d = 'Wednesday';
    ->     WHEN 4 THEN SET d = 'Thursday';
    ->     WHEN 5 THEN SET d = 'Friday';
    ->     WHEN 6 THEN SET d = 'Saturday';
    ->     WHEN 7 THEN SET d = 'Sunday';
    ->     ELSE SET d = 'Invalid day number';
    ->   END CASE;
    ->   SELECT n AS day_no, d AS day_name;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL day_of_week(3);
+--------+-----------+
| day_no | day_name  |
+--------+-----------+
|      3 | Wednesday |
+--------+-----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL day_of_week(7);
+--------+----------+
| day_no | day_name |
+--------+----------+
|      7 | Sunday   |
+--------+----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 15

```
mysql> CALL day_of_week(9);
+--------+--------------------+
| day_no | day_name           |
+--------+--------------------+
|      9 | Invalid day number |
+--------+--------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE grade_of(IN rno INT)
    -> BEGIN
    ->   DECLARE m INT;
    ->   DECLARE msg VARCHAR(40);
    ->   SELECT mark INTO m FROM student WHERE rollno = rno;
    ->   CASE
    ->     WHEN m >= 75 THEN SET msg = 'Distinction';
    ->     WHEN m >= 50 THEN SET msg = 'Pass';
    ->     ELSE SET msg = 'Fail';
    ->   END CASE;
    ->   SELECT rno AS rollno, m AS mark, msg AS result;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL grade_of(101);
+--------+------+-------------+
| rollno | mark | result      |
+--------+------+-------------+
|    101 |   92 | Distinction |
+--------+------+-------------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.00 sec)
```

MySQL Command Line - Ex. 15

```
mysql> CALL grade_of(104);
+--------+------+--------+
| rollno | mark | result |
+--------+------+--------+
|    104 |   55 | Pass   |
+--------+------+--------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.00 sec)

mysql> CALL grade_of(105);
+--------+------+--------+
| rollno | mark | result |
+--------+------+--------+
|    105 |   31 | Fail   |
+--------+------+--------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 15

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE print_numbers(IN n INT)
    -> BEGIN
    ->   DECLARE i INT DEFAULT 1;
    ->   DECLARE s VARCHAR(100) DEFAULT '';
    ->   lp: LOOP
    ->     IF i > n THEN
    ->       LEAVE lp;
    ->     END IF;
    ->     SET s = CONCAT(s, i, ' ');
    ->     SET i = i + 1;
    ->   END LOOP lp;
    ->   SELECT s AS numbers;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL print_numbers(10);
+----------------------+
| numbers              |
+----------------------+
| 1 2 3 4 5 6 7 8 9 10  |
+----------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 15

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE sum_even(IN n INT)
    -> BEGIN
    ->   DECLARE i INT DEFAULT 0;
    ->   DECLARE total INT DEFAULT 0;
    ->   lp: LOOP
    ->     SET i = i + 1;
    ->     IF i > n THEN
    ->       LEAVE lp;
    ->     END IF;
    ->     IF i MOD 2 <> 0 THEN
    ->       ITERATE lp;
    ->     END IF;
    ->     SET total = total + i;
    ->   END LOOP lp;
    ->   SELECT n AS upto, total AS sum_of_even;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL sum_even(10);
+------+-------------+
| upto | sum_of_even |
+------+-------------+
|   10 |          30 |
+------+-------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sum_even(20);
+------+-------------+
| upto | sum_of_even |
+------+-------------+
|   20 |         110 |
+------+-------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

Result:

Thus the CASE statement, CASE expression and LOOP (with LEAVE / ITERATE) were implemented and executed successfully.

Ex. No. 16

High Level Programming Extensions - Procedures

Aim: To create and execute stored procedures in MySQL using IN, OUT and INOUT parameters.

Output (MySQL Command Line):

MySQL Command Line - Ex. 16

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex16_procedures;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex16_procedures;
Database changed
mysql> CREATE TABLE employee (
    ->   emp_id INT PRIMARY KEY AUTO_INCREMENT,
    ->   emp_name VARCHAR(30),
    ->   dept VARCHAR(20),
    ->   salary DECIMAL(10,2));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO employee (emp_name, dept, salary) VALUES
    -> ('Ravi','IT',45000),('Meena','HR',38000),('Suresh','IT',52000),
    -> ('Divya','Finance',47000),('Karthik','HR',36000);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> DELIMITER //
mysql> CREATE PROCEDURE show_employees()
    -> BEGIN
    ->   SELECT * FROM employee ORDER BY emp_id;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 16

```
mysql> CALL show_employees();
+--------+----------+---------+----------+
| emp_id | emp_name | dept    | salary   |
+--------+----------+---------+----------+
|      1 | Ravi     | IT      | 45000.00 |
|      2 | Meena    | HR      | 38000.00 |
|      3 | Suresh   | IT      | 52000.00 |
|      4 | Divya    | Finance | 47000.00 |
|      5 | Karthik  | HR      | 36000.00 |
+--------+----------+---------+----------+
5 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 16

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE add_employee(IN p_name VARCHAR(30), IN p_dept VARCHAR(20), IN p_sal DECIMAL(10,2))
    -> BEGIN
    ->   INSERT INTO employee (emp_name, dept, salary) VALUES (p_name, p_dept, p_sal);
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL add_employee('Lakshmi','Finance',50000);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM employee;
+--------+----------+---------+----------+
| emp_id | emp_name | dept    | salary   |
+--------+----------+---------+----------+
|      1 | Ravi     | IT      | 45000.00 |
|      2 | Meena    | HR      | 38000.00 |
|      3 | Suresh   | IT      | 52000.00 |
|      4 | Divya    | Finance | 47000.00 |
|      5 | Karthik  | HR      | 36000.00 |
|      6 | Lakshmi  | Finance | 50000.00 |
+--------+----------+---------+----------+
6 rows in set (0.00 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE dept_stats(IN d VARCHAR(20), OUT cnt INT, OUT avg_sal DECIMAL(10,2))
    -> BEGIN
    ->   SELECT COUNT(*), AVG(salary) INTO cnt, avg_sal
    ->   FROM employee WHERE dept = d;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL dept_stats('IT', @c, @a);
Query OK, 1 row affected (0.00 sec)
```

MySQL Command Line - Ex. 16

```
mysql> SELECT @c AS emp_count, @a AS avg_salary;
+-----------+------------+
| emp_count | avg_salary |
+-----------+------------+
|         2 |   48500.00 |
+-----------+------------+
1 row in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 16

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE raise_salary(INOUT sal DECIMAL(10,2), IN pct INT)
    -> BEGIN
    ->   SET sal = sal + (sal * pct / 100);
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> SET @s = 40000;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL raise_salary(@s, 10);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @s AS new_salary;
+------------+
| new_salary |
+------------+
|   44000.00 |
+------------+
1 row in set (0.00 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE give_bonus(IN d VARCHAR(20), IN pct INT)
    -> BEGIN
    ->   UPDATE employee SET salary = salary + salary * pct / 100 WHERE dept = d;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL give_bonus('HR', 10);
Query OK, 2 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 16

```
mysql> SELECT emp_id, emp_name, dept, salary FROM employee WHERE dept = 'HR';
+--------+----------+------+----------+
| emp_id | emp_name | dept | salary   |
+--------+----------+------+----------+
|      2 | Meena    | HR   | 41800.00 |
|      5 | Karthik  | HR   | 39600.00 |
+--------+----------+------+----------+
2 rows in set (0.00 sec)

mysql> SELECT routine_name, routine_type FROM information_schema.routines
    -> WHERE routine_schema = 'ex16_procedures';
+---------------+--------------+
| routine_name  | routine_type |
+---------------+--------------+
| add_employee  | PROCEDURE    |
| dept_stats    | PROCEDURE    |
| give_bonus    | PROCEDURE    |
| raise_salary  | PROCEDURE    |
| show_employees | PROCEDURE    |
+---------------+--------------+
5 rows in set (0.00 sec)

mysql> DROP PROCEDURE show_employees;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT routine_name, routine_type FROM information_schema.routines
    -> WHERE routine_schema = 'ex16_procedures';
+-------------+--------------+
| routine_name | routine_type |
+-------------+--------------+
| add_employee | PROCEDURE    |
| dept_stats   | PROCEDURE    |
| give_bonus   | PROCEDURE    |
| raise_salary | PROCEDURE    |
+-------------+--------------+
4 rows in set (0.00 sec)

mysql>
```

Result:

Thus stored procedures with IN, OUT and INOUT parameters were created, called and dropped successfully.

Ex. No. 17

High Level Programming Extensions - Function

Aim: To create and use user-defined stored functions in MySQL and call them from SQL statements.

Output (MySQL Command Line):

MySQL Command Line - Ex. 17

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex17_functions;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex17_functions;
Database changed
mysql> CREATE TABLE product (pid INT PRIMARY KEY, pname VARCHAR(30), price DECIMAL(10,2), qty INT);
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO product VALUES (1,'Keyboard',850.00,10),(2,'Mouse',450.00,25),
    -> (3,'Monitor',9500.00,5),(4,'Pen Drive',600.00,40);
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> DELIMITER //
mysql> CREATE FUNCTION factorial(n INT) RETURNS BIGINT
    -> DETERMINISTIC
    -> BEGIN
    ->   DECLARE f BIGINT DEFAULT 1;
    ->   DECLARE i INT DEFAULT 1;
    ->   WHILE i <= n DO
    ->     SET f = f * i;
    ->     SET i = i + 1;
    ->   END WHILE;
    ->   RETURN f;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 17

```
mysql> SELECT factorial(5) AS fact_5, factorial(10) AS fact_10;
+--------+---------+
| fact_5 | fact_10 |
+--------+---------+
|    120 | 3628800 |
+--------+---------+
1 row in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 17

```
mysql> DELIMITER //
mysql> CREATE FUNCTION gst_amount(price DECIMAL(10,2), rate INT) RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->   RETURN price * rate / 100;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT pname, price, gst_amount(price,18) AS gst,
    ->        price + gst_amount(price,18) AS total
    -> FROM product;
+-----------+---------+---------+----------+
| pname     | price   | gst     | total    |
+-----------+---------+---------+----------+
| Keyboard  |  850.00 |  153.00 |  1003.00 |
| Mouse     |  450.00 |   81.00 |   531.00 |
| Monitor   | 9500.00 | 1710.00 | 11210.00 |
| Pen Drive |  600.00 |  108.00 |   708.00 |
+-----------+---------+---------+----------+
4 rows in set (0.00 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION grade_fn(m INT) RETURNS CHAR(4)
    -> DETERMINISTIC
    -> BEGIN
    ->   DECLARE g CHAR(4);
    ->   IF m >= 90 THEN SET g = 'O';
    ->   ELSEIF m >= 75 THEN SET g = 'A';
    ->   ELSEIF m >= 50 THEN SET g = 'B';
    ->   ELSE SET g = 'Fail';
    ->   END IF;
    ->   RETURN g;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 17

```
mysql> SELECT grade_fn(95) AS g1, grade_fn(78) AS g2, grade_fn(52) AS g3, grade_fn(30) AS g4;
+------+------+------+------+
| g1   | g2   | g3   | g4   |
+------+------+------+------+
| O    | A    | B    | Fail |
+------+------+------+------+
1 row in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 17

```
mysql> DELIMITER //
mysql> CREATE FUNCTION is_even(n INT) RETURNS VARCHAR(5)
    -> DETERMINISTIC
    -> BEGIN
    ->   IF n MOD 2 = 0 THEN
    ->     RETURN 'Even';
    ->   END IF;
    ->   RETURN 'Odd';
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT pname, qty, is_even(qty) AS qty_type FROM product;
+-----------+------+----------+
| pname     | qty  | qty_type |
+-----------+------+----------+
| Keyboard  |   10 | Even     |
| Mouse     |   25 | Odd      |
| Monitor   |    5 | Odd      |
| Pen Drive |   40 | Even     |
+-----------+------+----------+
4 rows in set (0.00 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION total_stock_value() RETURNS DECIMAL(12,2)
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE t DECIMAL(12,2);
    ->   SELECT SUM(price * qty) INTO t FROM product;
    ->   RETURN t;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 17

```
mysql> SELECT total_stock_value() AS stock_value;
+-------------+
| stock_value |
+-------------+
|    91250.00 |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT routine_name, routine_type, data_type FROM information_schema.routines
    -> WHERE routine_schema = 'ex17_functions';
+-------------------+--------------+-----------+
| routine_name      | routine_type | data_type |
+-------------------+--------------+-----------+
| factorial         | FUNCTION     | bigint    |
| grade_fn          | FUNCTION     | char      |
| gst_amount        | FUNCTION     | decimal   |
| is_even           | FUNCTION     | varchar   |
| total_stock_value | FUNCTION     | decimal   |
+-------------------+--------------+-----------+
5 rows in set (0.00 sec)

mysql> DROP FUNCTION is_even;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT is_even(4);
ERROR 1305 (42000): FUNCTION ex17_functions.is_even does not exist

mysql>
```

Result:

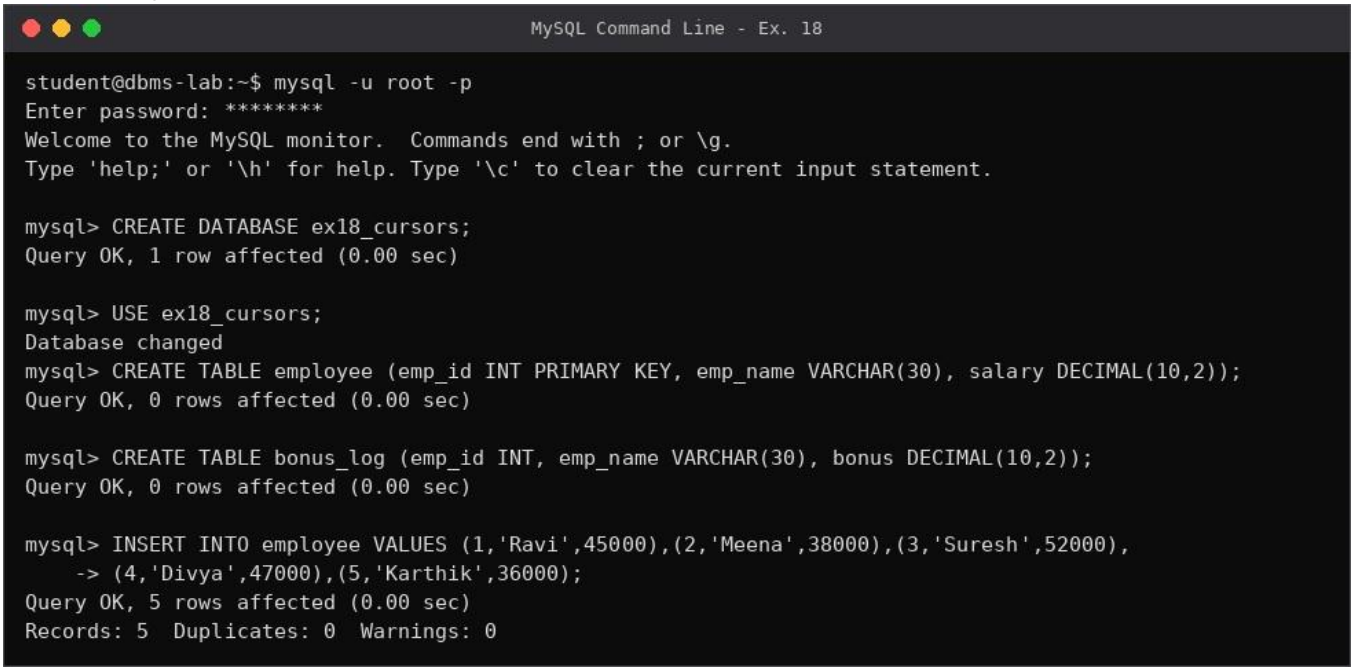
Thus user-defined functions were created and invoked from SELECT statements successfully.

Ex. No. 18

High Level Programming Extensions - Cursors

Aim: To declare, open, fetch from and close cursors inside stored procedures, using a NOT FOUND handler to detect the end of the result set.

Output (MySQL Command Line):



MySQL Command Line - Ex. 18

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE list_names()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE n VARCHAR(30);
    ->   DECLARE all_names VARCHAR(200) DEFAULT '';
    ->   DECLARE c1 CURSOR FOR SELECT emp_name FROM employee;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->   OPEN c1;
    ->   rd: LOOP
    ->     FETCH c1 INTO n;
    ->     IF done = 1 THEN
    ->       LEAVE rd;
    ->     END IF;
    ->     SET all_names = CONCAT(all_names, n, ', ');
    ->   END LOOP rd;
    ->   CLOSE c1;
    ->   SELECT TRIM(TRAILING ', ' FROM all_names) AS employees;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL list_names();
+-----------------------------------+
| employees                         |
+-----------------------------------+
| Ravi, Meena, Suresh, Divya, Karthik |
+-----------------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 18

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE calc_bonus()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE id INT;
    ->   DECLARE nm VARCHAR(30);
    ->   DECLARE sal DECIMAL(10,2);
    ->   DECLARE c2 CURSOR FOR SELECT emp_id, emp_name, salary FROM employee;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->   OPEN c2;
    ->   rd: LOOP
    ->     FETCH c2 INTO id, nm, sal;
    ->     IF done = 1 THEN
    ->       LEAVE rd;
    ->     END IF;
    ->     IF sal >= 45000 THEN
    ->       INSERT INTO bonus_log VALUES (id, nm, sal * 0.10);
    ->     ELSE
    ->       INSERT INTO bonus_log VALUES (id, nm, sal * 0.05);
    ->     END IF;
    ->   END LOOP rd;
    ->   CLOSE c2;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL calc_bonus();
Query OK, 5 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 18

```
mysql> SELECT * FROM bonus_log;
+--------+----------+---------+
| emp_id | emp_name | bonus   |
+--------+----------+---------+
|      1 | Ravi     | 4500.00 |
|      2 | Meena    | 1900.00 |
|      3 | Suresh   | 5200.00 |
|      4 | Divya    | 4700.00 |
|      5 | Karthik  | 1800.00 |
+--------+----------+---------+
5 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 18

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE total_salary(OUT total DECIMAL(12,2))
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE s DECIMAL(10,2);
    ->   DECLARE c3 CURSOR FOR SELECT salary FROM employee;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->   SET total = 0;
    ->   OPEN c3;
    ->   rd: LOOP
    ->     FETCH c3 INTO s;
    ->     IF done = 1 THEN
    ->       LEAVE rd;
    ->     END IF;
    ->     SET total = total + s;
    ->   END LOOP rd;
    ->   CLOSE c3;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL total_salary(@t);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @t AS total_salary;
+--------------+
| total_salary |
+--------------+
|    218000.00 |
+--------------+
1 row in set (0.00 sec)

mysql> SELECT SUM(salary) AS check_with_sum FROM employee;
+----------------+
| check_with_sum |
+----------------+
|      218000.00 |
+----------------+
1 row in set (0.00 sec)

mysql>
```

Result:

Thus cursors were declared, opened, fetched and closed inside stored procedures successfully.

Ex. No. 19

High Level Programming Extensions - Triggers

Aim: To create BEFORE and AFTER triggers for INSERT, UPDATE and DELETE events and to verify their automatic execution.

Output (MySQL Command Line):

MySQL Command Line - Ex. 19

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex19_triggers;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex19_triggers;
Database changed
mysql> CREATE TABLE account (acc_no INT PRIMARY KEY, holder VARCHAR(30), balance DECIMAL(10,2));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE audit_log (
    ->   log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   acc_no INT, action VARCHAR(20),
    ->   old_bal DECIMAL(10,2), new_bal DECIMAL(10,2),
    ->   log_time DATETIME);
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_before_insert BEFORE INSERT ON account
    -> FOR EACH ROW
    -> BEGIN
    ->   IF NEW.balance < 0 THEN
    ->     SET NEW.balance = 0;
    ->   END IF;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 19

```
mysql> DELIMITER //
mysql> CREATE TRIGGER trg_after_insert AFTER INSERT ON account
    -> FOR EACH ROW
    -> BEGIN
    ->   INSERT INTO audit_log (acc_no, action, old_bal, new_bal, log_time)
    ->   VALUES (NEW.acc_no, 'ACCOUNT OPENED', NULL, NEW.balance, NOW());
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_before_update BEFORE UPDATE ON account
    -> FOR EACH ROW
    -> BEGIN
    ->   IF NEW.balance < 0 THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient balance';
    ->   END IF;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 19

```
mysql> DELIMITER //
mysql> CREATE TRIGGER trg_after_update AFTER UPDATE ON account
    -> FOR EACH ROW
    -> BEGIN
    ->   INSERT INTO audit_log (acc_no, action, old_bal, new_bal, log_time)
    ->   VALUES (OLD.acc_no, 'BALANCE UPDATED', OLD.balance, NEW.balance, NOW());
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_after_delete AFTER DELETE ON account
    -> FOR EACH ROW
    -> BEGIN
    ->   INSERT INTO audit_log (acc_no, action, old_bal, new_bal, log_time)
    ->   VALUES (OLD.acc_no, 'ACCOUNT CLOSED', OLD.balance, NULL, NOW());
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT trigger_name, event_manipulation, event_object_table, action_timing
    -> FROM information_schema.triggers
    -> WHERE trigger_schema = 'ex19_triggers';
+-------------------+--------------------+--------------------+---------------+
| trigger_name      | event_manipulation | event_object_table | action_timing |
+-------------------+--------------------+--------------------+---------------+
| trg_before_insert | INSERT             | account            | BEFORE        |
| trg_after_insert  | INSERT             | account            | AFTER         |
| trg_before_update | UPDATE             | account            | BEFORE        |
| trg_after_update  | UPDATE             | account            | AFTER         |
| trg_after_delete  | DELETE             | account            | AFTER         |
+-------------------+--------------------+--------------------+---------------+
5 rows in set (0.00 sec)

mysql> INSERT INTO account VALUES (1001,'Anand',5000),(1002,'Bharathi',1500),(1003,'Chitra',-500);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

MySQL Command Line - Ex. 19

```
mysql> SELECT * FROM account;
+--------+----------+---------+
| acc_no | holder   | balance |
+--------+----------+---------+
|   1001 | Anand    | 5000.00 |
|   1002 | Bharathi | 1500.00 |
|   1003 | Chitra   |    0.00 |
+--------+----------+---------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM audit_log;
+--------+--------+----------------+---------+---------+---------------------+
| log_id | acc_no | action         | old_bal | new_bal | log_time            |
+--------+--------+----------------+---------+---------+---------------------+
|      1 |   1001 | ACCOUNT OPENED |    NULL | 5000.00 | 2026-10-24 07:10:04 |
|      2 |   1002 | ACCOUNT OPENED |    NULL | 1500.00 | 2026-10-24 07:10:04 |
|      3 |   1003 | ACCOUNT OPENED |    NULL |    0.00 | 2026-10-24 07:10:04 |
+--------+--------+----------------+---------+---------+---------------------+
3 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 19

```
mysql> UPDATE account SET balance = balance + 3000 WHERE acc_no = 1002;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE account SET balance = balance - 9000 WHERE acc_no = 1001;
ERROR 1644 (45000): Insufficient balance

mysql> DELETE FROM account WHERE acc_no = 1003;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM account;
+--------+----------+---------+
| acc_no | holder   | balance |
+--------+----------+---------+
|   1001 | Anand    | 5000.00 |
|   1002 | Bharathi | 4500.00 |
+--------+----------+---------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM audit_log;
+--------+--------+----------------+---------+---------+---------------------+
| log_id | acc_no | action         | old_bal | new_bal | log_time            |
+--------+--------+----------------+---------+---------+---------------------+
|      1 |   1001 | ACCOUNT OPENED |    NULL | 5000.00 | 2026-10-24 07:10:04 |
|      2 |   1002 | ACCOUNT OPENED |    NULL | 1500.00 | 2026-10-24 07:10:04 |
|      3 |   1003 | ACCOUNT OPENED |    NULL |    0.00 | 2026-10-24 07:10:04 |
|      4 |   1002 | BALANCE UPDATED | 1500.00 | 4500.00 | 2026-10-24 07:10:04 |
|      5 |   1003 | ACCOUNT CLOSED |    0.00 |    NULL | 2026-10-24 07:10:04 |
+--------+--------+----------------+---------+---------+---------------------+
5 rows in set (0.00 sec)

mysql> DROP TRIGGER trg_after_delete;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 19

```
mysql> SELECT trigger_name, event_manipulation, action_timing
    -> FROM information_schema.triggers
    -> WHERE trigger_schema = 'ex19_triggers';
+------------------+--------------------+---------------+
| trigger_name     | event_manipulation | action_timing |
+------------------+--------------------+---------------+
| trg_before_insert | INSERT            | BEFORE        |
| trg_after_insert  | INSERT            | AFTER         |
| trg_before_update | UPDATE            | BEFORE        |
| trg_after_update  | UPDATE            | AFTER         |
+------------------+--------------------+---------------+
4 rows in set (0.00 sec)

mysql>
```

Result:

Thus BEFORE and AFTER triggers on INSERT, UPDATE and DELETE were created and verified successfully.

Ex. No. 20

MySQL String Functions - REPLACE, REPEAT, REVERSE, RIGHT, LEFT, RPAD, LPAD

Aim: To use the string functions REPLACE, REPEAT, REVERSE, RIGHT, LEFT, RPAD and LPAD in MySQL queries.

Output (MySQL Command Line):

MySQL Command Line - Ex. 20

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex20_string_fn1;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex20_string_fn1;
Database changed
mysql> SELECT REPLACE('Welcome to DBMS Lab', 'DBMS', 'MySQL') AS replace_demo;
+----------------------+
| replace_demo         |
+----------------------+
| Welcome to MySQL Lab |
+----------------------+
1 row in set (0.00 sec)

mysql> SELECT REPEAT('DB-', 3) AS repeat_demo;
+-------------+
| repeat_demo |
+-------------+
| DB-DB-DB-   |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT REVERSE('MySQL') AS reverse_demo;
+--------------+
| reverse_demo |
+--------------+
| LQSyM        |
+--------------+
1 row in set (0.00 sec)
```

MySQL Command Line - Ex. 20

```
mysql> SELECT RIGHT('Database', 4) AS right_demo, LEFT('Database', 4) AS left_demo;
+------------+-----------+
| right_demo | left_demo |
+------------+-----------+
| base       | Data      |
+------------+-----------+
1 row in set (0.00 sec)

mysql> SELECT RPAD('Sql', 8, '*') AS rpad_demo, LPAD('7', 4, '0') AS lpad_demo;
+-----------+-----------+
| rpad_demo | lpad_demo |
+-----------+-----------+
| Sql*****  | 0007      |
+-----------+-----------+
1 row in set (0.00 sec)

mysql> SELECT LPAD('Hi', 6, '*') AS lpad_short, RPAD('Database', 4, '*') AS rpad_truncate;
+------------+---------------+
| lpad_short | rpad_truncate |
+------------+---------------+
| ****Hi     | Data          |
+------------+---------------+
1 row in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 20

```
mysql> CREATE TABLE customer (cid INT, cname VARCHAR(20), city VARCHAR(20), phone CHAR(10));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO customer VALUES (1,'Aravind','Chennai','9840012345'),
    -> (2,'Bhavana','Madurai','9841123456'),(3,'Charan','Chennai','9942234567');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT cname, REPLACE(city,'Chennai','Madras') AS old_name FROM customer;
+---------+----------+
| cname   | old_name |
+---------+----------+
| Aravind | Madras   |
| Bhavana | Madurai  |
| Charan  | Madras   |
+---------+----------+
3 rows in set (0.00 sec)

mysql> SELECT cname, LEFT(cname,3) AS short_name, RIGHT(phone,4) AS last4 FROM customer;
+---------+------------+-------+
| cname   | short_name | last4 |
+---------+------------+-------+
| Aravind | Ara        | 2345  |
| Bhavana | Bha        | 3456  |
| Charan  | Cha        | 4567  |
+---------+------------+-------+
3 rows in set (0.00 sec)

mysql> SELECT LPAD(cid,5,'0') AS cust_code, RPAD(cname,12,'.') AS padded_name FROM customer;
+-----------+--------------+
| cust_code | padded_name  |
+-----------+--------------+
| 00001     | Aravind..... |
| 00002     | Bhavana..... |
| 00003     | Charan...... |
+-----------+--------------+
3 rows in set (0.00 sec)
```

MySQL Command Line - Ex. 20

```
mysql> SELECT cname, CONCAT(LEFT(phone,2), REPEAT('X',6), RIGHT(phone,2)) AS masked_phone
    -> FROM customer;
+---------+--------------+
| cname   | masked_phone |
+---------+--------------+
| Aravind | 98XXXXXX45   |
| Bhavana | 98XXXXXX56   |
| Charan  | 99XXXXXX67   |
+---------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT word, IF(word = REVERSE(word), 'Palindrome', 'Not palindrome') AS result
    -> FROM (SELECT 'madam' AS word UNION SELECT 'mysql' UNION SELECT 'level') t;
+-------+----------------+
| word  | result         |
+-------+----------------+
| madam | Palindrome     |
| mysql | Not palindrome |
| level | Palindrome     |
+-------+----------------+
3 rows in set (0.00 sec)

mysql>
```

Result:

Thus the string functions REPLACE, REPEAT, REVERSE, RIGHT, LEFT, RPAD and LPAD were executed successfully.

Ex. No. 21

MySQL String Functions - SPACE, SUBSTR, UPPER, LOWER, TRIM, LENGTH

Aim: To use the string functions SPACE, SUBSTR, UPPER, LOWER, TRIM and LENGTH in MySQL queries.

Output (MySQL Command Line):

MySQL Command Line - Ex. 21

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex21_string_fn2;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex21_string_fn2;
Database changed
mysql> SELECT CONCAT('Hello', SPACE(5), 'World') AS space_demo;
+-----------------+
| space_demo      |
+-----------------+
| Hello     World |
+-----------------+
1 row in set (0.00 sec)

mysql> SELECT SUBSTR('Database Management', 1, 8) AS part1,
    ->        SUBSTR('Database Management', 10) AS part2,
    ->        SUBSTR('Database Management', -6, 3) AS part3;
+----------+------------+-------+
| part1    | part2      | part3 |
+----------+------------+-------+
| Database | Management | gem   |
+----------+------------+-------+
1 row in set (0.00 sec)

mysql> SELECT UPPER('mysql lab') AS upper_demo, LOWER('MYSQL LAB') AS lower_demo;
+------------+------------+
| upper_demo | lower_demo |
+------------+------------+
| MYSQL LAB  | mysql lab  |
+------------+------------+
1 row in set (0.00 sec)
```

MySQL Command Line - Ex. 21

```
mysql> SELECT TRIM('   MySQL   ') AS both_sides,
    ->        TRIM(LEADING 'x' FROM 'xxMySQLxx') AS leading_x,
    ->        TRIM(TRAILING 'x' FROM 'xxMySQLxx') AS trailing_x;
+------------+-----------+------------+
| both_sides | leading_x | trailing_x |
+------------+-----------+------------+
| MySQL      | MySQLxx   | xxMySQL    |
+------------+-----------+------------+
1 row in set (0.00 sec)

mysql> SELECT LENGTH('Database') AS len1, LENGTH('  MySQL  ') AS len_with_spaces,
    ->        LENGTH(TRIM('  MySQL  ')) AS len_after_trim;
+------+-----------------+----------------+
| len1 | len_with_spaces | len_after_trim |
+------+-----------------+----------------+
|    8 |               9 |              5 |
+------+-----------------+----------------+
1 row in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 21

```
mysql> CREATE TABLE employee (eid INT, ename VARCHAR(30), email VARCHAR(40));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO employee VALUES (1,'  ravi kumar ','Ravi.Kumar@Company.com'),
    -> (2,' meena s','MEENA.S@company.com'),(3,'suresh  ','suresh@Company.com');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT eid, ename, LENGTH(ename) AS len_raw, LENGTH(TRIM(ename)) AS len_trim
    -> FROM employee;
+------+---------------+---------+----------+
| eid  | ename         | len_raw | len_trim |
+------+---------------+---------+----------+
|    1 |   ravi kumar  |      13 |       10 |
|    2 |  meena s      |       8 |        7 |
|    3 | suresh        |       8 |        6 |
+------+---------------+---------+----------+
3 rows in set (0.00 sec)

mysql> SELECT eid, UPPER(TRIM(ename)) AS name_upper, LOWER(email) AS email_lower
    -> FROM employee;
+------+------------+-----------------------+
| eid  | name_upper | email_lower           |
+------+------------+-----------------------+
|    1 | RAVI KUMAR | ravi.kumar@company.com |
|    2 | MEENA S    | meena.s@company.com    |
|    3 | SURESH     | suresh@company.com     |
+------+------------+-----------------------+
3 rows in set (0.00 sec)
```

MySQL Command Line - Ex. 21

```
mysql> SELECT eid, SUBSTR(email, 1, LOCATE('@',email) - 1) AS username,
    ->        SUBSTR(email, LOCATE('@',email) + 1) AS domain
    -> FROM employee;
+------+------------+-------------+
| eid  | username   | domain      |
+------+------------+-------------+
|    1 | Ravi.Kumar | Company.com |
|    2 | MEENA.S    | company.com |
|    3 | suresh     | Company.com |
+------+------------+-------------+
3 rows in set (0.00 sec)

mysql> SELECT CONCAT(UPPER(SUBSTR(TRIM(ename),1,1)), SPACE(2), SUBSTR(TRIM(ename),2)) AS formatted
    -> FROM employee;
+--------------+
| formatted    |
+--------------+
| R  avi kumar |
| M  eena s    |
| S  uresh     |
+--------------+
3 rows in set (0.00 sec)

mysql>
```

Result:

Thus the string functions SPACE, SUBSTR, UPPER, LOWER, TRIM and LENGTH were executed successfully.

Ex. No. 22

Database Connectivity Using PHP and MySQL

Aim: To connect a PHP program to a MySQL database using the mysqli extension and to perform INSERT, SELECT, UPDATE and DELETE operations.

Output (MySQL Command Line):

MySQL Command Line - Ex. 22

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex22_php_mysql;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex22_php_mysql;
Database changed
mysql> CREATE TABLE student (
    ->   id INT PRIMARY KEY AUTO_INCREMENT,
    ->   name VARCHAR(30), dept VARCHAR(10), cgpa DECIMAL(4,2));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO student (name, dept, cgpa) VALUES ('Arun','CSE',8.75),('Bala','IT',8.10);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> CREATE USER 'labuser'@'localhost' IDENTIFIED BY 'lab@123';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL PRIVILEGES ON ex22_php_mysql.* TO 'labuser'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

student@dbms-lab: ~/php

```
student@dbms-lab:~/php$ cat connect.php
<?php
$host = "localhost";
$user = "labuser";
$pass = "lab@123";
$db   = "ex22_php_mysql";

$conn = mysqli_connect($host, $user, $pass, $db);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully to database: $db\n";
mysqli_close($conn);
?>

student@dbms-lab:~/php$ php connect.php
Connected successfully to database: ex22_php_mysql

student@dbms-lab:~/php$
```

student@dbms-lab: ~/php

```
student@dbms-lab:~/php$ cat insert_select.php
<?php
$conn = mysqli_connect("localhost", "labuser", "lab@123", "ex22_php_mysql");
if (!$conn) { die("Connection failed: " . mysqli_connect_error()); }

// INSERT a new record
$sql = "INSERT INTO student (name, dept, cgpa) VALUES ('Nandhini', 'ECE', 8.45)";
if (mysqli_query($conn, $sql)) {
    echo "New record inserted. ID = " . mysqli_insert_id($conn) . "\n";
}

// SELECT all records
$result = mysqli_query($conn, "SELECT * FROM student");
echo "Rows found: " . mysqli_num_rows($result) . "\n";
printf("%-4s %-10s %-6s %s\n", "ID", "NAME", "DEPT", "CGPA");
while ($row = mysqli_fetch_assoc($result)) {
    printf("%-4s %-10s %-6s %s\n", $row['id'], $row['name'], $row['dept'], $row['cgpa']);
}
mysqli_close($conn);
?>

student@dbms-lab:~/php$ php insert_select.php
New record inserted. ID = 3
Rows found: 3
ID   NAME       DEPT   CGPA
1    Arun       CSE    8.75
2    Bala       IT     8.10
3    Nandhini   ECE    8.45

student@dbms-lab:~/php$
```

student@dbms-lab: ~/php

```
student@dbms-lab:~/php$ cat update_delete.php
<?php
$conn = mysqli_connect("localhost", "labuser", "lab@123", "ex22_php_mysql");

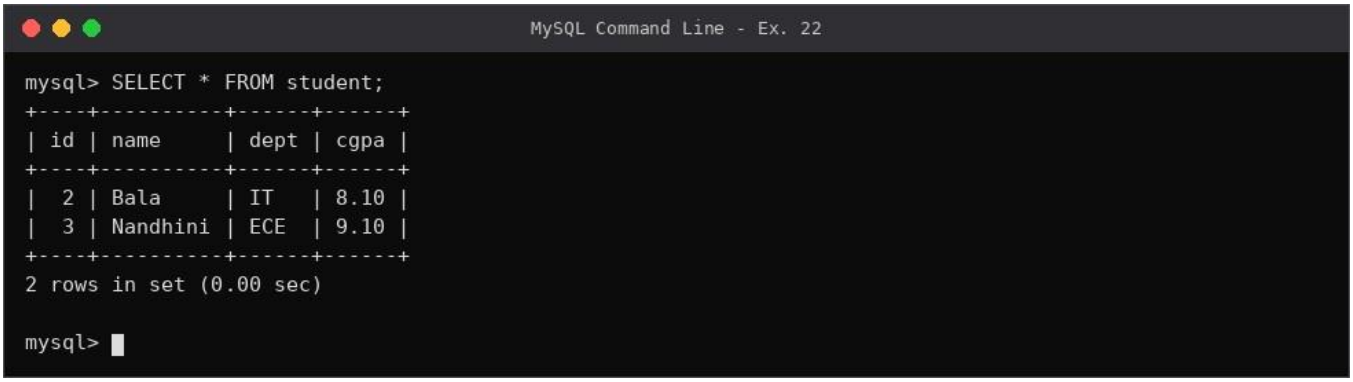
mysqli_query($conn, "UPDATE student SET cgpa = 9.10 WHERE name = 'Nandhini'");
echo "Updated rows: " . mysqli_affected_rows($conn) . "\n";

mysqli_query($conn, "DELETE FROM student WHERE id = 1");
echo "Deleted rows: " . mysqli_affected_rows($conn) . "\n";

$r = mysqli_query($conn, "SELECT COUNT(*) AS total FROM student");
$row = mysqli_fetch_assoc($r);
echo "Records now in table: " . $row['total'] . "\n";
mysqli_close($conn);
?>

student@dbms-lab:~/php$ php update_delete.php
Updated rows: 1
Deleted rows: 1
Records now in table: 2

student@dbms-lab:~/php$
```



Result:

Thus a PHP program was connected to the MySQL database and the INSERT, SELECT, UPDATE and DELETE operations were performed successfully.

Ex. No. 23

Case Scenario: Train Ticket Reservation System to Receive Tickets through Social Network

Aim: To design a MySQL database for a train ticket reservation system in which the confirmed e-ticket is delivered to the passenger through a social network (WhatsApp / Facebook / Telegram / Twitter) and to implement the required queries.

Output (MySQL Command Line):

MySQL Command Line - Ex. 23

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex23_train_ticket;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex23_train_ticket;
Database changed
mysql> CREATE TABLE train (
    ->   train_no INT PRIMARY KEY, train_name VARCHAR(25),
    ->   source VARCHAR(15), destination VARCHAR(15),
    ->   dep_time TIME, fare DECIMAL(8,2), seats INT);
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE passenger (
    ->   pid INT PRIMARY KEY AUTO_INCREMENT, pname VARCHAR(25), age INT,
    ->   mobile CHAR(10), social_id VARCHAR(20),
    ->   platform ENUM('WhatsApp','Facebook','Telegram','Twitter'));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE ticket (
    ->   pnr BIGINT PRIMARY KEY, pid INT, train_no INT,
    ->   journey_date DATE, seat_no VARCHAR(8),
    ->   status ENUM('CONFIRMED','WAITING','CANCELLED'), amount DECIMAL(8,2),
    ->   FOREIGN KEY (pid) REFERENCES passenger(pid),
    ->   FOREIGN KEY (train_no) REFERENCES train(train_no));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE ticket_share (
    ->   share_id INT PRIMARY KEY AUTO_INCREMENT, pnr BIGINT,
    ->   platform VARCHAR(10), shared_to VARCHAR(20),
    ->   sent_status ENUM('SENT','FAILED'), sent_time DATETIME,
    ->   FOREIGN KEY (pnr) REFERENCES ticket(pnr));
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 23

```
mysql> SHOW TABLES;
+-----------------------------+
| Tables_in_ex23_train_ticket |
+-----------------------------+
| passenger                   |
| ticket                      |
| ticket_share                |
| train                       |
+-----------------------------+
4 rows in set (0.00 sec)

mysql> DESC ticket;
+--------------+-----------------------------------------+------+-----+---------+-------+
| Field        | Type                                    | Null | Key | Default | Extra |
+--------------+-----------------------------------------+------+-----+---------+-------+
| pnr          | bigint(20)                              | NO   | PRI | NULL    |       |
| pid          | int(11)                                 | YES  | MUL | NULL    |       |
| train_no     | int(11)                                 | YES  | MUL | NULL    |       |
| journey_date | date                                    | YES  |     | NULL    |       |
| seat_no      | varchar(8)                              | YES  |     | NULL    |       |
| status       | enum('CONFIRMED','WAITING','CANCELLED') | YES  |     | NULL    |       |
| amount       | decimal(8,2)                            | YES  |     | NULL    |       |
+--------------+-----------------------------------------+------+-----+---------+-------+
7 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 23

```
mysql> INSERT INTO train VALUES
    -> (12635,'Vaigai Express','Chennai','Madurai','06:30:00',385.00,5),
    -> (12631,'Nellai Express','Chennai','Tirunelveli','19:40:00',520.00,4),
    -> (12605,'Pallavan Express','Chennai','Trichy','15:45:00',310.00,3);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO passenger (pname,age,mobile,social_id,platform) VALUES
    -> ('Arjun',28,'9840012345','+91 98400 12345','WhatsApp'),
    -> ('Priya',24,'9841123456','priya.k.fb','Facebook'),
    -> ('Karthik',35,'9942234567','@karthik_t','Telegram'),
    -> ('Meena',31,'9444345678','@meena_s','Twitter'),
    -> ('Suresh',42,'9884456789','+91 98844 56789','WhatsApp');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO ticket VALUES
    -> (8201234501,1,12635,'2026-11-05','S1-23','CONFIRMED',385.00),
    -> (8201234502,2,12635,'2026-11-05','S1-24','CONFIRMED',385.00),
    -> (8201234503,3,12631,'2026-11-06','B2-11','CONFIRMED',520.00),
    -> (8201234504,4,12605,'2026-11-06','S3-07','CONFIRMED',310.00),
    -> (8201234505,5,12631,'2026-11-06','WL-01','WAITING',520.00);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> UPDATE train t SET seats = seats - (SELECT COUNT(*) FROM ticket k
    ->   WHERE k.train_no = t.train_no AND k.status = 'CONFIRMED');
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> SELECT train_no, train_name, source, destination, seats FROM train;
+----------+------------------+---------+-------------+-------+
| train_no | train_name       | source  | destination | seats |
+----------+------------------+---------+-------------+-------+
|    12605 | Pallavan Express | Chennai | Trichy      |     2 |
|    12631 | Nellai Express   | Chennai | Tirunelveli |     3 |
|    12635 | Vaigai Express   | Chennai | Madurai     |     3 |
+----------+------------------+---------+-------------+-------+
3 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 23

```
mysql> INSERT INTO ticket_share (pnr, platform, shared_to, sent_status, sent_time)
    -> SELECT k.pnr, p.platform, p.social_id, 'SENT', NOW()
    -> FROM ticket k JOIN passenger p ON k.pid = p.pid
    -> WHERE k.status = 'CONFIRMED';
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT share_id, pnr, platform, shared_to, sent_status FROM ticket_share;
+----------+------------+----------+-----------------+-------------+
| share_id | pnr        | platform | shared_to       | sent_status |
+----------+------------+----------+-----------------+-------------+
|        1 | 8201234501 | WhatsApp | +91 98400 12345 | SENT        |
|        2 | 8201234502 | Facebook | priya.k.fb      | SENT        |
|        3 | 8201234503 | Telegram | @karthik_t      | SENT        |
|        4 | 8201234504 | Twitter  | @meena_s        | SENT        |
+----------+------------+----------+-----------------+-------------+
4 rows in set (0.00 sec)

mysql> SELECT p.pname, p.platform,
    ->   CONCAT('PNR ',k.pnr,' ',tr.train_name,' ',k.journey_date,' ',k.seat_no) AS ticket_message
    -> FROM ticket k JOIN passenger p ON k.pid = p.pid
    -> JOIN train tr ON k.train_no = tr.train_no
    -> WHERE k.status = 'CONFIRMED';
+---------+----------+-------------------------------------------------+
| pname   | platform | ticket_message                                  |
+---------+----------+-------------------------------------------------+
| Meena   | Twitter  | PNR 8201234504 Pallavan Express 2026-11-06 S3-07 |
| Karthik | Telegram | PNR 8201234503 Nellai Express 2026-11-06 B2-11   |
| Arjun   | WhatsApp | PNR 8201234501 Vaigai Express 2026-11-05 S1-23   |
| Priya   | Facebook | PNR 8201234502 Vaigai Express 2026-11-05 S1-24   |
+---------+----------+-------------------------------------------------+
4 rows in set (0.00 sec)

mysql> UPDATE ticket_share SET sent_status = 'FAILED' WHERE platform = 'Telegram';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

MySQL Command Line - Ex. 23

```
mysql> SELECT share_id, pnr, platform, sent_status FROM ticket_share WHERE sent_status = 'FAILED';
+----------+------------+----------+-------------+
| share_id | pnr        | platform | sent_status |
+----------+------------+----------+-------------+
|        3 | 8201234503 | Telegram | FAILED      |
+----------+------------+----------+-------------+
1 row in set (0.00 sec)

mysql> UPDATE ticket_share SET sent_status = 'SENT', sent_time = NOW()
    -> WHERE sent_status = 'FAILED';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
```

MySQL Command Line - Ex. 23

```
mysql> SELECT platform, COUNT(*) AS tickets_sent FROM ticket_share
    -> WHERE sent_status = 'SENT' GROUP BY platform;
+----------+--------------+
| platform | tickets_sent |
+----------+--------------+
| Facebook |            1 |
| Telegram |            1 |
| Twitter  |            1 |
| WhatsApp |            1 |
+----------+--------------+
4 rows in set (0.00 sec)

mysql> UPDATE ticket SET status = 'CANCELLED' WHERE pnr = 8201234504;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE train SET seats = seats + 1 WHERE train_no = 12605;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT tr.train_name, COUNT(*) AS bookings, SUM(k.amount) AS revenue
    -> FROM ticket k JOIN train tr ON k.train_no = tr.train_no
    -> WHERE k.status = 'CONFIRMED' GROUP BY tr.train_name;
+----------------+----------+---------+
| train_name     | bookings | revenue |
+----------------+----------+---------+
| Nellai Express |        1 |  520.00 |
| Vaigai Express |        2 |  770.00 |
+----------------+----------+---------+
2 rows in set (0.00 sec)

mysql> CREATE VIEW v_share_report AS
    -> SELECT p.pname, k.pnr, s.platform, s.sent_status
    -> FROM ticket_share s JOIN ticket k ON s.pnr = k.pnr
    -> JOIN passenger p ON k.pid = p.pid;
Query OK, 0 rows affected (0.00 sec)
```

MySQL Command Line - Ex. 23

```
mysql> SELECT * FROM v_share_report;
+---------+------------+----------+-------------+
| pname   | pnr        | platform | sent_status |
+---------+------------+----------+-------------+
| Arjun   | 8201234501 | WhatsApp | SENT        |
| Priya   | 8201234502 | Facebook | SENT        |
| Karthik | 8201234503 | Telegram | SENT        |
| Meena   | 8201234504 | Twitter  | SENT        |
+---------+------------+----------+-------------+
4 rows in set (0.00 sec)

mysql>
```

Result:

Thus the train ticket reservation system with ticket delivery through social networks was implemented and the queries were executed successfully.

Ex. No. 24

Case Scenario: College Admission Form

Aim: To design a MySQL database for an online college admission form, process the applications by merit (HSC percentage) and generate the admission reports.

Output (MySQL Command Line):

MySQL Command Line - Ex. 24

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex24_college_admission;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex24_college_admission;
Database changed
mysql> CREATE TABLE course (
    ->   course_id CHAR(4) PRIMARY KEY, course_name VARCHAR(30),
    ->   seats INT, cutoff DECIMAL(5,2));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE applicant (
    ->   app_no INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(25), dob DATE,
    ->   gender CHAR(1), community VARCHAR(5), mobile CHAR(10), email VARCHAR(35),
    ->   course_id CHAR(4), hsc_percent DECIMAL(5,2),
    ->   applied_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ->   FOREIGN KEY (course_id) REFERENCES course(course_id)) AUTO_INCREMENT = 1001;
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE admission (
    ->   adm_no INT PRIMARY KEY AUTO_INCREMENT, app_no INT UNIQUE,
    ->   course_id CHAR(4), rank_no INT,
    ->   status ENUM('SELECTED','WAITLISTED','REJECTED','WITHDRAWN'),
    ->   FOREIGN KEY (app_no) REFERENCES applicant(app_no));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO course VALUES ('CSE','B.E. Computer Science',3,85.00),
    -> ('ECE','B.E. Electronics & Comm.',2,80.00),('MECH','B.E. Mechanical',2,75.00);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

MySQL Command Line - Ex. 24

```
mysql> SELECT * FROM course;
+-----------+--------------------------+-------+--------+
| course_id | course_name              | seats | cutoff |
+-----------+--------------------------+-------+--------+
| CSE       | B.E. Computer Science    |     3 |  85.00 |
| ECE       | B.E. Electronics & Comm. |     2 |  80.00 |
| MECH      | B.E. Mechanical          |     2 |  75.00 |
+-----------+--------------------------+-------+--------+
3 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 24

```
mysql> INSERT INTO applicant (name,dob,gender,community,mobile,email,course_id,hsc_percent) VALUES
    -> ('Aravind','2008-03-14','M','OC','9840011111','aravind@mail.com','CSE',96.40),
    -> ('Bhavana','2008-07-21','F','BC','9841122222','bhavana@mail.com','CSE',93.20),
    -> ('Charan','2007-12-02','M','MBC','9942233333','charan@mail.com','CSE',91.75),
    -> ('Deepika','2008-01-30','F','OC','9444455555','deepika@mail.com','CSE',88.50),
    -> ('Eswar','2008-05-11','M','BC','9884466666','eswar@mail.com','ECE',90.10),
    -> ('Fathima','2008-09-09','F','BC','9790077777','fathima@mail.com','ECE',87.60),
    -> ('Gokul','2007-11-25','M','SC','9500188888','gokul@mail.com','ECE',82.30),
    -> ('Harini','2008-02-17','F','OC','9600199999','harini@mail.com','MECH',80.25),
    -> ('Imran','2008-06-06','M','BC','9710100000','imran@mail.com','MECH',77.80),
    -> ('Janani','2008-08-19','F','MBC','9840122222','janani@mail.com','MECH',72.40);
Query OK, 10 rows affected (0.00 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT app_no, name, course_id, hsc_percent FROM applicant;
+--------+---------+-----------+-------------+
| app_no | name    | course_id | hsc_percent |
+--------+---------+-----------+-------------+
|   1001 | Aravind | CSE       |       96.40 |
|   1002 | Bhavana | CSE       |       93.20 |
|   1003 | Charan  | CSE       |       91.75 |
|   1004 | Deepika | CSE       |       88.50 |
|   1005 | Eswar   | ECE       |       90.10 |
|   1006 | Fathima | ECE       |       87.60 |
|   1007 | Gokul   | ECE       |       82.30 |
|   1008 | Harini  | MECH      |       80.25 |
|   1009 | Imran   | MECH      |       77.80 |
|   1010 | Janani  | MECH      |       72.40 |
+--------+---------+-----------+-------------+
10 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 24

```
mysql> SELECT app_no, name, course_id, hsc_percent,
    ->   RANK() OVER (PARTITION BY course_id ORDER BY hsc_percent DESC) AS merit_rank
    -> FROM applicant ORDER BY course_id, merit_rank;
+--------+---------+-----------+-------------+------------+
| app_no | name    | course_id | hsc_percent | merit_rank |
+--------+---------+-----------+-------------+------------+
|   1001 | Aravind | CSE       |       96.40 |          1 |
|   1002 | Bhavana | CSE       |       93.20 |          2 |
|   1003 | Charan  | CSE       |       91.75 |          3 |
|   1004 | Deepika | CSE       |       88.50 |          4 |
|   1005 | Eswar   | ECE       |       90.10 |          1 |
|   1006 | Fathima | ECE       |       87.60 |          2 |
|   1007 | Gokul   | ECE       |       82.30 |          3 |
|   1008 | Harini  | MECH      |       80.25 |          1 |
|   1009 | Imran   | MECH      |       77.80 |          2 |
|   1010 | Janani  | MECH      |       72.40 |          3 |
+--------+---------+-----------+-------------+------------+
10 rows in set (0.00 sec)

mysql> INSERT INTO admission (app_no, course_id, rank_no, status)
    -> SELECT a.app_no, a.course_id, a.rn,
    ->   CASE WHEN a.hsc_percent < c.cutoff THEN 'REJECTED'
    ->        WHEN a.rn <= c.seats THEN 'SELECTED'
    ->        ELSE 'WAITLISTED' END
    -> FROM (SELECT app_no, course_id, hsc_percent,
    ->   RANK() OVER (PARTITION BY course_id ORDER BY hsc_percent DESC) AS rn
    ->   FROM applicant) a JOIN course c ON a.course_id = c.course_id;
Query OK, 10 rows affected (0.00 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql>
```

MySQL Command Line - Ex. 24

```
mysql> SELECT a.app_no, a.name, c.course_name, ad.rank_no, ad.status
    -> FROM admission ad JOIN applicant a ON ad.app_no = a.app_no
    -> JOIN course c ON ad.course_id = c.course_id
    -> ORDER BY c.course_id, ad.rank_no;
+--------+---------+-------------------------+---------+------------+
| app_no | name    | course_name             | rank_no | status     |
+--------+---------+-------------------------+---------+------------+
|   1001 | Aravind | B.E. Computer Science   |       1 | SELECTED   |
|   1002 | Bhavana | B.E. Computer Science   |       2 | SELECTED   |
|   1003 | Charan  | B.E. Computer Science   |       3 | SELECTED   |
|   1004 | Deepika | B.E. Computer Science   |       4 | WAITLISTED |
|   1005 | Eswar   | B.E. Electronics & Comm. |       1 | SELECTED   |
|   1006 | Fathima | B.E. Electronics & Comm. |       2 | SELECTED   |
|   1007 | Gokul   | B.E. Electronics & Comm. |       3 | WAITLISTED |
|   1008 | Harini  | B.E. Mechanical         |       1 | SELECTED   |
|   1009 | Imran   | B.E. Mechanical         |       2 | SELECTED   |
|   1010 | Janani  | B.E. Mechanical         |       3 | REJECTED   |
+--------+---------+-------------------------+---------+------------+
10 rows in set (0.00 sec)

mysql> SELECT course_id, status, COUNT(*) AS candidates
    -> FROM admission GROUP BY course_id, status;
+-----------+------------+------------+
| course_id | status     | candidates |
+-----------+------------+------------+
| CSE       | SELECTED   |          3 |
| CSE       | WAITLISTED |          1 |
| ECE       | SELECTED   |          2 |
| ECE       | WAITLISTED |          1 |
| MECH      | SELECTED   |          2 |
| MECH      | REJECTED   |          1 |
+-----------+------------+------------+
6 rows in set (0.00 sec)

mysql> UPDATE admission SET status = 'WITHDRAWN' WHERE app_no = 1002;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

MySQL Command Line - Ex. 24

```
mysql> UPDATE admission SET status = 'SELECTED' WHERE app_no = 1004;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT c.course_id, c.seats,
    ->   COUNT(CASE WHEN ad.status = 'SELECTED' THEN 1 END) AS filled,
    ->   c.seats - COUNT(CASE WHEN ad.status = 'SELECTED' THEN 1 END) AS vacant
    -> FROM course c LEFT JOIN admission ad ON c.course_id = ad.course_id
    -> GROUP BY c.course_id, c.seats;
+-----------+-------+--------+--------+
| course_id | seats | filled | vacant |
+-----------+-------+--------+--------+
| CSE       |     3 |      3 |      0 |
| ECE       |     2 |      2 |      0 |
| MECH      |     2 |      2 |      0 |
+-----------+-------+--------+--------+
3 rows in set (0.00 sec)

mysql> SELECT a.app_no, a.name, a.dob, a.gender, a.community, a.course_id,
    ->   a.hsc_percent, ad.status FROM applicant a
    -> JOIN admission ad ON a.app_no = ad.app_no WHERE a.app_no = 1003\G
*************************** 1. row ***************************
     app_no: 1003
       name: Charan
        dob: 2007-12-02
     gender: M
  community: MBC
  course_id: CSE
hsc_percent: 91.75
     status: SELECTED
1 row in set (0.00 sec)

mysql>
```

Result:

Thus the college admission database was created and the application, merit ranking and allotment queries were executed successfully.

Ex. No. 25

Case Scenario: QR Enabled Automatic Bus Ticket Booking System

Aim: To design a MySQL database for a QR enabled automatic bus ticket booking system in which every booking generates a unique QR code that is validated once when the passenger boards.

Output (MySQL Command Line):

MySQL Command Line - Ex. 25

```
student@dbms-lab:~$ mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE ex25_qr_bus;
Query OK, 1 row affected (0.00 sec)

mysql> USE ex25_qr_bus;
Database changed
mysql> CREATE TABLE route (
    ->   route_id INT PRIMARY KEY, source VARCHAR(15), destination VARCHAR(15),
    ->   distance_km INT, fare DECIMAL(8,2));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE bus (
    ->   bus_id INT PRIMARY KEY, bus_no VARCHAR(14), route_id INT,
    ->   total_seats INT, available_seats INT, dep_time TIME,
    ->   FOREIGN KEY (route_id) REFERENCES route(route_id));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE passenger (pid INT PRIMARY KEY, pname VARCHAR(25), mobile CHAR(10));
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE booking (
    ->   booking_id INT PRIMARY KEY AUTO_INCREMENT, pid INT, bus_id INT,
    ->   journey_date DATE, seats INT, amount DECIMAL(8,2),
    ->   qr_code VARCHAR(20) UNIQUE,
    ->   status ENUM('BOOKED','USED','CANCELLED'),
    ->   FOREIGN KEY (pid) REFERENCES passenger(pid),
    ->   FOREIGN KEY (bus_id) REFERENCES bus(bus_id));
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO route VALUES (1,'Chennai','Kanchipuram',72,80.00),
    -> (2,'Chennai','Vellore',135,150.00),(3,'Chennai','Pondicherry',150,170.00);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

MySQL Command Line - Ex. 25

```
mysql> INSERT INTO bus VALUES (1,'TN01-AB-1234',1,40,40,'06:00:00'),
    -> (2,'TN01-CD-5678',2,40,40,'07:30:00'),(3,'TN09-EF-9012',3,30,30,'09:00:00');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO passenger VALUES (1,'Arun','9840011223'),(2,'Bhuvana','9841122334'),
    -> (3,'Chandru','9942233445'),(4,'Divya','9444455667');
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
```

MySQL Command Line - Ex. 25

```
mysql> CREATE TRIGGER trg_seat_reduce AFTER INSERT ON booking
    -> FOR EACH ROW
    -> UPDATE bus SET available_seats = available_seats - NEW.seats
    -> WHERE bus_id = NEW.bus_id;
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_seat_restore AFTER UPDATE ON booking
    -> FOR EACH ROW
    -> BEGIN
    ->   IF NEW.status = 'CANCELLED' AND OLD.status = 'BOOKED' THEN
    ->     UPDATE bus SET available_seats = available_seats + NEW.seats
    ->     WHERE bus_id = NEW.bus_id;
    ->   END IF;
    -> END//
mysql> DELIMITER ;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO booking (pid,bus_id,journey_date,seats,amount,qr_code,status) VALUES
    -> (1,1,'2026-11-02',2,160.00,UPPER(SUBSTR(MD5('B1-P1-2026-11-02'),1,12)),'BOOKED'),
    -> (2,2,'2026-11-02',1,150.00,UPPER(SUBSTR(MD5('B2-P2-2026-11-02'),1,12)),'BOOKED'),
    -> (3,3,'2026-11-03',3,510.00,UPPER(SUBSTR(MD5('B3-P3-2026-11-03'),1,12)),'BOOKED'),
    -> (4,1,'2026-11-02',1,80.00,UPPER(SUBSTR(MD5('B1-P4-2026-11-02'),1,12)),'BOOKED');
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT b.booking_id, p.pname, bs.bus_no, b.seats, b.qr_code, b.status
    -> FROM booking b JOIN passenger p ON b.pid = p.pid
    -> JOIN bus bs ON b.bus_id = bs.bus_id;
+------------+---------+--------------+-------+--------------+--------+
| booking_id | pname   | bus_no       | seats | qr_code      | status |
+------------+---------+--------------+-------+--------------+--------+
|          1 | Arun    | TN01-AB-1234 |     2 | DA6A10095A8D | BOOKED |
|          2 | Bhuvana | TN01-CD-5678 |     1 | E6983D647ACA | BOOKED |
|          3 | Chandru | TN09-EF-9012 |     3 | 9A5B431B68EC | BOOKED |
|          4 | Divya   | TN01-AB-1234 |     1 | 0769AA1DB56D | BOOKED |
+------------+---------+--------------+-------+--------------+--------+
4 rows in set (0.00 sec)
```

MySQL Command Line - Ex. 25

```
mysql> SELECT bus_no, total_seats, available_seats FROM bus;
+--------------+-------------+-----------------+
| bus_no       | total_seats | available_seats |
+--------------+-------------+-----------------+
| TN01-AB-1234 |          40 |              37 |
| TN01-CD-5678 |          40 |              39 |
| TN09-EF-9012 |          30 |              27 |
+--------------+-------------+-----------------+
3 rows in set (0.00 sec)

mysql>
```

MySQL Command Line - Ex. 25

```
mysql> SELECT b.booking_id, p.pname, b.seats, b.status FROM booking b
    -> JOIN passenger p ON b.pid = p.pid WHERE b.qr_code = 'DA6A10095A8D';
+------------+-------+-------+--------+
| booking_id | pname | seats | status |
+------------+-------+-------+--------+
|          1 | Arun  |     2 | BOOKED |
+------------+-------+-------+--------+
1 row in set (0.00 sec)

mysql> UPDATE booking SET status = 'USED'
    -> WHERE qr_code = 'DA6A10095A8D' AND status = 'BOOKED';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE booking SET status = 'USED'
    -> WHERE qr_code = 'DA6A10095A8D' AND status = 'BOOKED';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> SELECT booking_id, qr_code, status FROM booking WHERE qr_code = 'INVALID12345';
Empty set (0.00 sec)

mysql> UPDATE booking SET status = 'CANCELLED' WHERE booking_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT bus_no, total_seats, available_seats FROM bus;
+--------------+-------------+-----------------+
| bus_no       | total_seats | available_seats |
+--------------+-------------+-----------------+
| TN01-AB-1234 |          40 |              37 |
| TN01-CD-5678 |          40 |              40 |
| TN09-EF-9012 |          30 |              27 |
+--------------+-------------+-----------------+
3 rows in set (0.00 sec)
```

MySQL Command Line - Ex. 25

```
mysql> SELECT r.source, r.destination, SUM(b.amount) AS revenue
    -> FROM booking b JOIN bus bs ON b.bus_id = bs.bus_id
    -> JOIN route r ON bs.route_id = r.route_id
    -> WHERE b.status IN ('BOOKED','USED')
    -> GROUP BY r.source, r.destination;
+---------+-------------+---------+
| source  | destination | revenue |
+---------+-------------+---------+
| Chennai | Kanchipuram |  240.00 |
| Chennai | Pondicherry |  510.00 |
+---------+-------------+---------+
2 rows in set (0.00 sec)

mysql> SELECT booking_id, qr_code, status FROM booking;
+------------+--------------+-----------+
| booking_id | qr_code      | status    |
+------------+--------------+-----------+
|          1 | DA6A10095A8D | USED      |
|          2 | E6983D647ACA | CANCELLED |
|          3 | 9A5B431B68EC | BOOKED    |
|          4 | 0769AA1DB56D | BOOKED    |
+------------+--------------+-----------+
4 rows in set (0.00 sec)

mysql>
```

Result:

Thus the QR enabled bus ticket booking database was implemented and the booking, QR validation and reporting queries were executed successfully.